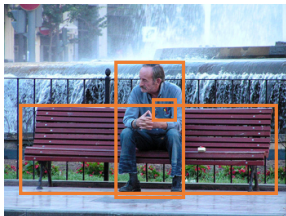


Model	Overall	Count	Exist	Belong	Query
NS-VQA	87.7	83.3	91.5	91.1	86.4
NS-CL	93.3	91.3	95.6	93.9	94.3

Table 10: Our model achieves comparable results on the Minecraft dataset with baselines trained by full program annotations.

G.2 VQS DATASET

We conduct experiments on the VQS dataset (Gan et al., 2017). VQS is a subset of the VQA 1.0 dataset (Antol et al., 2015). It contains questions that can be visually grounded: each question is associated with multiple image regions, annotated by humans as necessary for answering the question.



Q: Does this man have any pens on him?

A: Yes.

P: Exist(Filter(Man,
Relate(Have, Filter(Pen))
))

Figure 10: An example image from the VQS dataset. The orange bounding boxes are object proposals. On the right, we show the original question and answer in natural language, as well as the latent program recovered by our parser. To answer this question, models are expected to attend to the man and his pen in the pocket.

Setup. All models are trained on the first 63,509 images of the training set, and tested on the test split. For hyper-parameter tuning and model selection, the rest 5,000 images from the training set are used for validation. We use the multiple-choice setup for VQA: the models choose their most confident answer from 18 candidate answers for each question.

To obtain the latent programs from natural languages, we use a pre-trained syntactic dependency parser (Andreas et al., 2016; Schuster et al., 2015) for extracting programs and concepts that need to be learned. A sample question and the program obtained by our parser is shown in Figure 10. The concept embeddings are initialized by the bag of words (BoW) over the GloVe word embeddings (Pennington et al., 2014).

Baselines. We compare our model against two representative baselines: MLP (Jabri et al., 2016) and MAC (Hudson & Manning, 2018).

MLP is a standard baseline for visual-question answering, which treats the multiple-choice task as a ranking problem. For a specific candidate answer, a multi-layer perceptron (MLP) model is used to encode a tuple of the image, the question, and the candidate answer. The MLP outputs a score for each tuple, and the answer to the question is the candidate with the highest score. We encode the image with a ResNet-34 pre-trained on ImageNet and use BoW over the GloVe word embeddings for the question and option encoding.

We slightly modify the MAC network for the VQS dataset. For each candidate answer, we concatenate the question and the answer as the input to the model. The MAC model outputs a score from 0 to 1 and the answer to the question is the candidate with the highest score. The image features are extracted from the same ResNet-34 model.

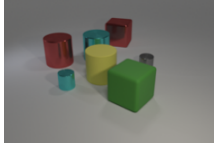
Results. Table 7 summarizes the results. NS-CL achieves comparable results with the MLP baseline and the MAC network designed for visual reasoning. Our model also brings transparent reasoning over natural images and language. Example execution traces generated by NS-CL are shown in Figure 13. Besides, the symbolic reasoning process helps us to inspect the model and diagnose the error sources. See the caption for details.

H VISUALIZATION OF EXECUTION TRACES AND VISUAL CONCEPTS

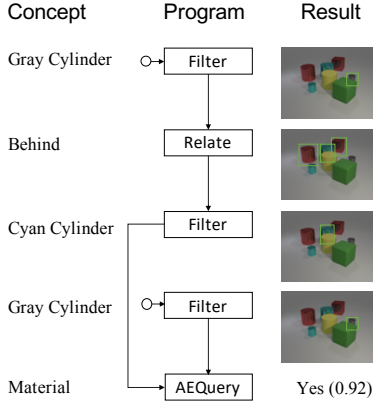
Another appealing benefit is that our reasoning model enjoys full interpretability. Figure 11, Figure 12, and Figure 13 show NS-CL’s execution traces on CLEVR, Minecraft, and VQS, respectively. As a side product, our system detects ambiguous and invalid programs and throws out exceptions. As an example (Figure 11), the question “What’s the color of the cylinder?” can be ambiguous if there are multiple cylinders or even invalid if there are no cylinders.

Figure 14 and Figure 15 include qualitative visualizations of the concepts learned from the CLEVR and Minecraft datasets, including object categories, attributes, and relations. We choose samples from the validation or test split of each dataset by generating queries of the corresponding concepts. We set a threshold to filter the returned images and objects. For quantitative evaluations of the learned concepts on the CLEVR dataset, please refer to Table 2 and Table 5.

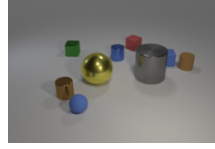
Example A.



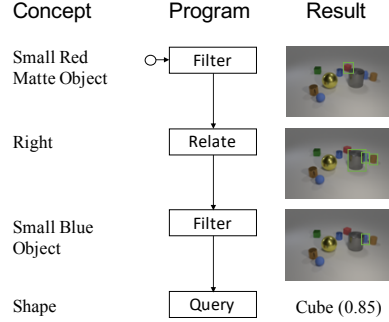
Q: Do the cyan cylinder that is behind the gray cylinder and the gray cylinder have the same material?



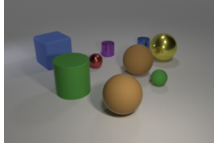
Example B.



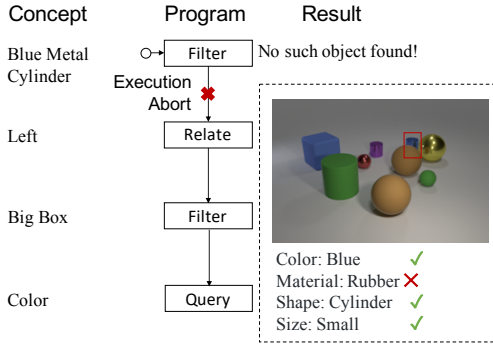
Q: There is a small blue object that is to the right of the small red matte object; what shape is it?



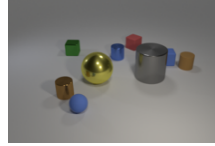
Example C. Failure Case



Q: What is the color of the big box left of the blue metal cylinder?



Example D. Ambiguous Program Case



Q: What is the color of the big metal object?

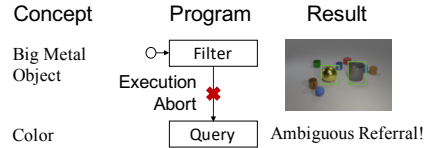


Figure 11: Visualization of the execution trace generated by our Neuro-Symbolic Concept Learner on the CLEVR dataset. Example A and B are successful executions that generate correct answers. In example C, the execution aborts at the first operator. To inspect the reason why the execution engine fails to find the corresponding object, we can read out the visual representation of the object, and locate the error source as the misclassification of the object material. Example D shows how our symbolic execution engine can detect invalid or ambiguous programs during the execution by performing sanity checks.